

POS TECH

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 3 | AULA 03 -

IMPLEMENTAÇÃO DE RESILIÊNCIA EM JAVA

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3

HANDS ON 4

SAIBA MAIS 5

O QUE VOCÊ VIU NESTA AULA? 15

REFERÊNCIAS 16

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Na aula anterior, exploramos a importância da resiliência em sistemas distribuídos e estratégias para aplicá-la de forma eficaz. Aprendemos conceitos e dicas práticas para entender e adotar essas estratégias em diferentes contextos.

Agora, vamos dar um passo à frente e aprender a implementar o padrão Circuit Breaker com exemplos práticos de código. Você verá como a abordagem pode proteger seu sistema contra falhas, garantindo maior estabilidade e disponibilidade. Vamos começar?

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

Nessa videoaula, vamos explorar padrões de resiliência utilizando a biblioteca Resilience4j. Veremos, na prática, como implementar padrões como Retry, Bulkhead e Rate Limiter em projetos Java.

Além de mostrar a aplicação desses padrões no código, compartilharemos dicas sobre configurações, boas práticas e como aproveitar ao máximo a documentação oficial para facilitar sua implementação e customização.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

Na aula anterior, exploramos os conceitos de padrões de resiliência em sistemas distribuídos. Discutimos estratégias como Circuit Breaker, Bulkhead, Rate Limiting, Retry, entre outros, explicando como esses padrões ajudam a criar sistemas mais robustos, minimizando impactos de falhas e garantindo maior estabilidade. Aprendemos que a resiliência não elimina falhas, mas reduz os efeitos delas.

O Resilience4j foi criado por Robert Winkler como uma alternativa moderna, leve e modular ao Netflix Hystrix, que era amplamente utilizado para implementar padrões de resiliência, mas foi descontinuado em 2018. Enquanto o Hystrix estava intimamente ligado à arquitetura baseada em threads e ao modelo do Spring Boot, o Resilience4j foi projetado para ser mais flexível, aproveitando as melhorias do Java 8+ e simplificando a integração com bibliotecas e frameworks.

O Resilience4j foi criado devido à necessidade de uma solução que fosse leve e modular, permitindo que pessoas desenvolvedoras utilizassem apenas os módulos necessários, como Circuit Breaker, Retry ou Bulkhead, sem carregar dependências desnecessárias.

O Resilience4j é compatível com padrões modernos e totalmente funcional com Java 8 e suas características, como Lambdas e Optional. Projetado para trabalhar de forma eficiente com frameworks como Spring Boot, mas também independente, podendo ser usado em qualquer projeto Java. Vamos nos aprofundar na prática, focando na **implementação de resiliência em Java,** usando ferramentas e bibliotecas como o Resilience4j.

Comparativo: Resilience4j vs Netflix Hystrix

O Netflix Hystrix foi pioneiro na implementação de padrões de resiliência, como o Circuit Breaker. O Resilience4j surgiu como seu sucessor natural, trazendo melhorias significativas para atender às necessidades de sistemas modernos e arquiteturas baseadas em Java 8+.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Aspecto	Resilience4j	Netflix Hystrix
Arquitetura Modular	Cada funcionalidade (Circuit Breaker, Retry, Bulkhead, etc.) é um módulo independente. Você só usa o que precisa.	Todas as funcionalidades vêm integradas, aumentando o peso da biblioteca mesmo que não sejam usadas.
Abordagem de Chamadas	Usa decorators para melhorar métodos, lambdas ou referências de método com padrões como Circuit Breaker.	Requer que chamadas sejam encapsuladas em objetos como HystrixCommand ou HystrixObservableCommand.
Combinação de Padrões	Permite combinar múltiplos padrões, como Bulkhead, Retry, Rate Limiter e Circuit Breaker, na mesma chamada.	Não suporta a combinação de padrões diretamente; cada funcionalidade precisa ser configurada separadamente.
Estados do Circuit Breaker	Permite múltiplas chamadas configuráveis no estado "meio aberto" para verificar se o sistema se recuperou.	Apenas uma chamada é permitida no estado "meio aberto" para decidir se o circuito deve fechar.
Chamadas Assíncronas	Oferece suporte nativo para execução assíncrona com CompletableFuture ou integração com RxJava e Reactor.	Suporte limitado para chamadas assíncronas, com dependência de HystrixObservableCommand.
Detecção de Chamadas Lentas	Pode abrir o Circuit Breaker com base em limiares configuráveis de tempo de resposta, mesmo antes de exceções serem lançadas.	Não possui suporte nativo para detectar e reagir a chamadas lentas.
Métricas e Monitoramento	Emite eventos ricos em informações, como resultados de execução e latência, facilmente integrados com ferramentas modernas de monitoramento.	Também emite eventos, mas com menos flexibilidade e integração direta limitada com ferramentas modernas.
Atualidade do Projeto	Projeto ativo, com suporte e atualizações frequentes para atender às necessidades de arquiteturas modernas.	Projeto descontinuado, sem suporte oficial ou novas atualizações desde 2018.

Figura 1 – Resilience4j vs Netflix Hystrix
Fonte: elaborado pelo autor (2024)

Vantagens do Resilience4j sobre o Hystrix

Quando falamos em vantagens do uso do Resilience4j em relação ao Hystrix, podemos citar a modularidade como um dos grandes diferenciais. Ela permite que você escolha apenas os padrões de resiliência que deseja implementar, como Circuit Breaker, Retry ou Bulkhead, reduzindo tanto a complexidade quanto o consumo de memória da aplicação. No caso do Hystrix, todos os padrões vêm integrados, mesmo que você precise apenas de um deles, o que pode ser desnecessário para algumas situações.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

A flexibilidade do Resilience4j é evidente no uso de decorators. Diferente do Hystrix, que exige o encapsulamento de chamadas em classes específicas, o Resilience4j permite decorar métodos, lambdas ou referências de métodos dinamicamente. Essa abordagem se alinha bem ao paradigma funcional introduzido no Java 8 e oferece maior simplicidade e adaptabilidade.

Outro ponto forte do Resilience4j é o suporte nativo para execução assíncrona por meio de CompletableFuture, RxJava ou Reactor, sem necessidade de dependências adicionais. Essa funcionalidade torna o framework mais moderno e preparado para aplicações que exigem alta performance.

A detecção de chamadas lentas é outro recurso avançado do Resilience4j. Ele possibilita abrir o Circuit Breaker antes que o sistema remoto fique completamente inativo, baseando-se no tempo de resposta das chamadas. Essa detecção proativa é algo que o Hystrix não oferece, limitando sua eficiência em alguns cenários críticos.

O propósito do Circuit Breaker

O Circuit Breaker foi introduzido como um padrão de design para resolver problemas em sistemas distribuídos, onde serviços falhos e lentos podem impactar negativamente toda a aplicação. O objetivo principal é evitar falhas em cascata, permitindo que o sistema degrade graciosamente e continue operando, mesmo que alguns componentes estejam indisponíveis.

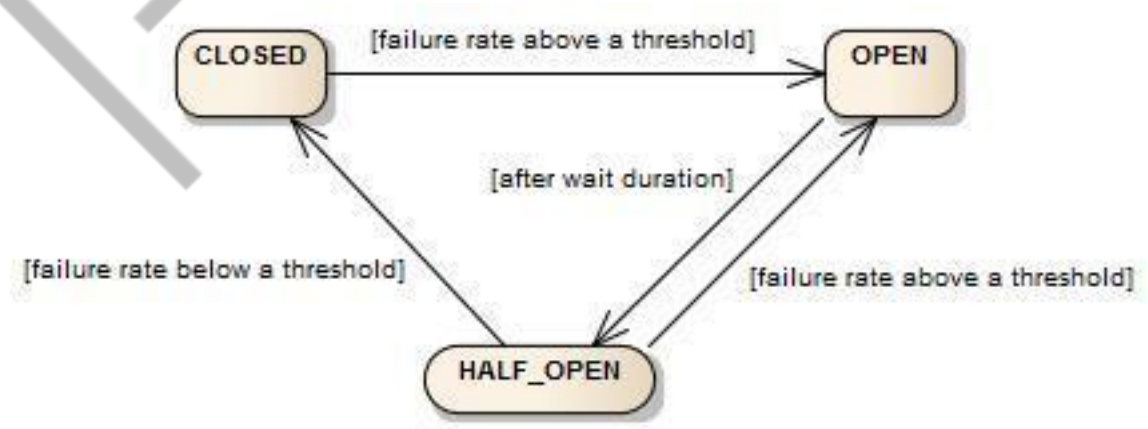


Figura 2 - Circuitbreak
Fonte: resilience4j.readme.io (2024)

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O Circuit Breaker utiliza o conceito de janela deslizante para monitorar e agregar os resultados das chamadas realizadas. Essa janela pode ser configurada com base em contagem ou em tempo, dependendo da sua necessidade.

Na janela deslizante baseada em contagem, o sistema mantém um número fixo de medições em um array circular. Por exemplo, se o tamanho da janela for 10, ela sempre armazenará os dados das últimas 10 chamadas ou tentativas. Cada vez que uma nova chamada é registrada, a soma total das medições é atualizada, adicionando o novo valor e subtraindo o valor mais antigo, que é removido da janela.

Esse processo é conhecido como Subtract-on-Evict. Como a soma é continuamente ajustada, calcular um resumo das medições é extremamente rápido, com complexidade constante O(1)O(1). A memória utilizada depende diretamente do tamanho da janela, sendo proporcional a este número (O(n)O(n)).

Já na janela deslizante baseada em tempo, o monitoramento é feito com buckets, que são agregações parciais organizadas em um array circular. Para uma janela de 10 segundos, por exemplo, o array conterá 10 buckets, sendo que cada bucket armazenará os resultados das chamadas realizadas em um segundo específico.

O bucket correspondente ao segundo atual é constantemente atualizado com os resultados das chamadas em andamento, enquanto os demais buckets refletem os dados de segundos passados. Assim como na abordagem por contagem, quando um bucket mais antigo é descartado, sua agregação parcial é subtraída da soma total e o bucket é reiniciado, também utilizando o Subtract-on-Evict.

Cada bucket não armazena os dados de cada chamada individualmente, mas mantém agregações simples, como o número de chamadas com falha, o número de chamadas lentas, o total de chamadas realizadas e a soma da duração total de todas as chamadas no bucket.

Essas técnicas garantem que o Circuit Breaker seja eficiente tanto no uso de memória quanto na velocidade de cálculo, tornando-o uma solução escalável e prática para gerenciar a resiliência em sistemas distribuídos.

Agora, vem a pergunta: como implementamos cada um deles?

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Primeiramente, precisamos ter as dependências no projeto e escolher como utilizar cada uma das abordagens. Se você estiver utilizando Maven, basta adicionar a seguinte configuração no seu arquivo pom.xml. Depois, teremos dois exemplos.

```
<dependency>
        <groupId>io.github.resilience4j</groupId>
         <artifactId>resilience4j-circuitbreaker</artifactId>
          <version>2.0.2</version>
</dependency>
```

Código-fonte 1 – Exemplo de código-fonte Maven
Fonte: elaborado pelo autor (2024)

Circuit Breaker com janela baseada em contagem

Aqui configuramos um Circuit Breaker com janela deslizante baseada em contagem. Um exemplo de configuração no **application.yml** para analisarmos:

```yaml
resilience4j:
  circuitbreaker:
    instances:
      countBasedCircuitBreaker:
        slidingWindowType: COUNT_BASED
        slidingWindowSize: 10 # Número de chamadas monitoradas
        failureRateThreshold: 50 # Abre o circuito se 50% das chamadas falharem
        waitDurationInOpenState: 5s # Tempo antes de tentar reabrir o circuito
```

Figura 3 – Exemplo de código-fonte YML
Fonte: elaborado pelo autor (2024)

```
import org.springframework.stereotype.Service;

import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;


@Service

public class CountBasedService {


@CircuitBreaker(name = "countBasedCircuitBreaker", fallbackMethod = "fallback")

public String processRequest() {
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
// Simula uma chamada que pode falhar

if (Math.random() > 0.5) {

throw new RuntimeException("Erro na chamada simulada");

}

return "Processamento bem-sucedido";

}


// Método de fallback para lidar com falhas

public String fallback(Throwable throwable) {

return "Fallback ativado devido a: " + throwable.getMessage();

}

}
```

Código-fonte 2 – Exemplo de código-fonte Java
Fonte: elaborado pelo autor (2024)

Nesta abordagem, o Circuit Breaker monitora um número fixo de chamadas recentes para decidir se o circuito deve permanecer fechado (funcionando normalmente), aberto (bloqueando chamadas) ou em estado de meio-aberto (testando se o sistema se recuperou).

Sobre os parâmetros:

slidingWindowType: define o tipo de janela deslizante. Quando configurado como COUNT_BASED, a janela monitora um número fixo de chamadas.

slidingWindowSize: determina quantas chamadas serão monitoradas. Por exemplo, se slidingWindowSize for 10, o Circuit Breaker analisará as últimas 10 chamadas realizadas.

failureRateThreshold: define a porcentagem de falhas que, ao ser atingida, abrirá o circuito. Por exemplo, 50.0 significa que o circuito abrirá se metade das chamadas monitoradas falharem.

waitDurationInOpenState: tempo que o Circuit Breaker permanece aberto antes de tentar transitar para o estado de meio-aberto. Por exemplo, 5s significa que ele aguardará 5 segundos antes de reavaliar.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Como funciona:

Quando o Circuit Breaker detecta que o percentual de falhas dentro das últimas N chamadas (tamanho da janela) excede o limite configurado, ele entra no estado **aberto**, bloqueando novas chamadas temporariamente.

Circuit Breaker com janela baseada em tempo

Aqui, monitoramos as chamadas realizadas dentro de uma janela de tempo fixa. O circuito abre ou fecha com base nos resultados acumulados nesse intervalo.

Aqui configuramos um Circuit Breaker com janela deslizante baseada em contagem. Abre seu arquivo de configuração, **application.yml**

```yaml
resilience4j:
  circuitbreaker:
    instances:
      timeBasedCircuitBreaker:
        slidingWindowType: TIME_BASED
        slidingWindowSize: 10s # Janela de 10 segundos
        failureRateThreshold: 50 # Abre o circuito se 50% das chamadas falharem
        waitDurationInOpenState: 5s # Tempo antes de tentar reabrir o circuito
```

Figura 4 – Exemplo de código-fonte YML
Fonte: elaborado pelo autor (2024)

```
import org.springframework.stereotype.Service;

import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;


@Service

public class TimeBasedService {


@CircuitBreaker(name = "timeBasedCircuitBreaker", fallbackMethod = "fallback")

public String processRequest() {

// Simula uma chamada que pode falhar

if (Math.random() > 0.5) {

throw new RuntimeException("Erro na chamada simulada");
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
}

return "Processamento bem-sucedido";

}


// Método de fallback para lidar com falhas

public String fallback(Throwable throwable) {

return "Fallback ativado devido a: " + throwable.getMessage();

}

}
```

Código-fonte 3 – Exemplo de código-fonte JAVA
Fonte: elaborado pelo autor (2024)

Nesta abordagem, o Circuit Breaker monitora as chamadas realizadas em um intervalo de tempo fixo. Essa configuração é ideal para cenários onde a análise de desempenho deve ser feita considerando o comportamento de um serviço ao longo de um período.

Parâmetros:

slidingWindowType: configurado como TIME_BASED para monitorar chamadas realizadas dentro de um intervalo de tempo.

slidingWindowSize: especifica a duração da janela em segundos. Por exemplo, se configurado como 10s, o Circuit Breaker analisará as chamadas feitas nos últimos 10 segundos.

failureRateThreshold: define o limite de falhas aceitáveis, em termos percentuais. Por exemplo, 50.0 significa que o circuito será aberto se metade das chamadas feitas nos últimos 10 segundos falharem.

waitDurationInOpenState: tempo que o Circuit Breaker permanece aberto antes de tentar transitar para o estado de meio-aberto, como 5s.

Como funciona:

Ao monitorar um intervalo de tempo específico, o Circuit Breaker atualiza os resultados agregados continuamente, descartando os dados mais antigos e

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

adicionando novos. Quando o percentual de falhas ultrapassa o limite, ele abre o circuito.

Métodos de Fallback

O método de fallback é uma estratégia de contingência para lidar com falhas. Ele é chamado automaticamente quando o Circuit Breaker está aberto ou uma exceção é lançada. O fallback permite retornar uma resposta padrão para o cliente, executar um caminho alternativo para recuperar dados e logar a falha e alertar os administradores. O parâmetro fallbackMethod vai definir qual método será chamado para lidar com a falha. Ele precisa receber como parâmetro a exceção capturada ou nenhum parâmetro, dependendo da lógica, lembrando que o estado do Circuit Breaker opera em três estados principais:

Fechado: chamadas funcionam normalmente e são monitoradas.

Aberto: chamadas são bloqueadas e as falhas são registradas.

Meio-aberto: após o tempo configurado, uma chamada é permitida para testar se o sistema se recuperou. Se for bem-sucedida, o circuito retorna ao estado fechado.

Uma boa prática quando configuramos o Circuit Breaker em um sistema é de que ele esteja ajustado às necessidades reais da aplicação. Uma das práticas mais importantes é definir limites realistas. Utilize dados históricos do sistema ou faça testes de performance para analisar o tempo de respostas e taxas de falhas, para ajustar parâmetros como failureRateThreshold e slowCallRateThreshold. Isso evita que o Circuit Breaker seja acionado desnecessariamente ou, ao contrário, seja tão permissivo que perca sua funcionalidade principal.

Outra boa prática é combinar o Circuit Breaker com o padrão Bulkhead. Isso ajuda a proteger os recursos locais, como threads e conexões, garantindo que, mesmo quando o Circuit Breaker estiver "Fechado", falhas em um serviço específico não sobrecarreguem outras partes do sistema. A combinação desses padrões aumenta significativamente a resiliência da aplicação.

É importante também implementar fallbacks úteis. Certifique-se de que as mensagens de erro ou as ações alternativas forneçam informações claras ou soluções práticas, como redirecionar chamadas para serviços de backup ou retornar valores

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

padrão. Isso não apenas melhora a experiência do usuário, mas também ajuda a mitigar os impactos de falhas nos sistemas interdependentes.

Por fim, monitore regularmente o comportamento do Circuit Breaker, acompanhando as métricas em tempo real, como o número de chamadas bem-sucedidas, falhas e lentas. O monitoramento contínuo permite ajustar as configurações conforme o sistema evolui, garantindo que ele continue eficiente e adaptado às novas demandas.

Na próxima aula, sobre Monitoramento e Escalabilidade, vamos abordar estratégias e ferramentas que permitam acompanhar o desempenho de sistemas distribuídos em tempo real, garantindo que eles evoluam com eficiência e mantenham alta disponibilidade. Esperamos por você!

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Você aprendeu a implementar o Circuit Breaker com Java, utilizando duas técnicas essenciais para tornar seu sistema mais resiliente e robusto. Durante a aula, exploramos na prática como aplicar com biblioteca Resilience4j, com exemplos que facilitam o entendimento e a implementação. Não se esqueça de assistir os vídeos desta aula para consolidar o aprendizado e aprimorar ainda mais suas habilidades!

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

JSESSION4D. **A Quick Guide to Resilience4j with Spring Boot.** 2023. Disponível em: https://jsession4d.com/a-quick-guide-to-resilience4j-with-spring-boot/. Acesso em: 13 dez. 2024.

LYDTECH Consulting. **Resilience4j Overview.** 2023. Disponível em: https://www.lydtechconsulting.com/blog-resilience4j.html. Acesso em: 13 dez. 2024.

RESILIENCE4J. **Circuit Breaker Documentation.** 2023. Disponível em: https://resilience4j.readme.io/docs/circuitbreaker. Acesso em: 13 dez. 2024.

FIAP

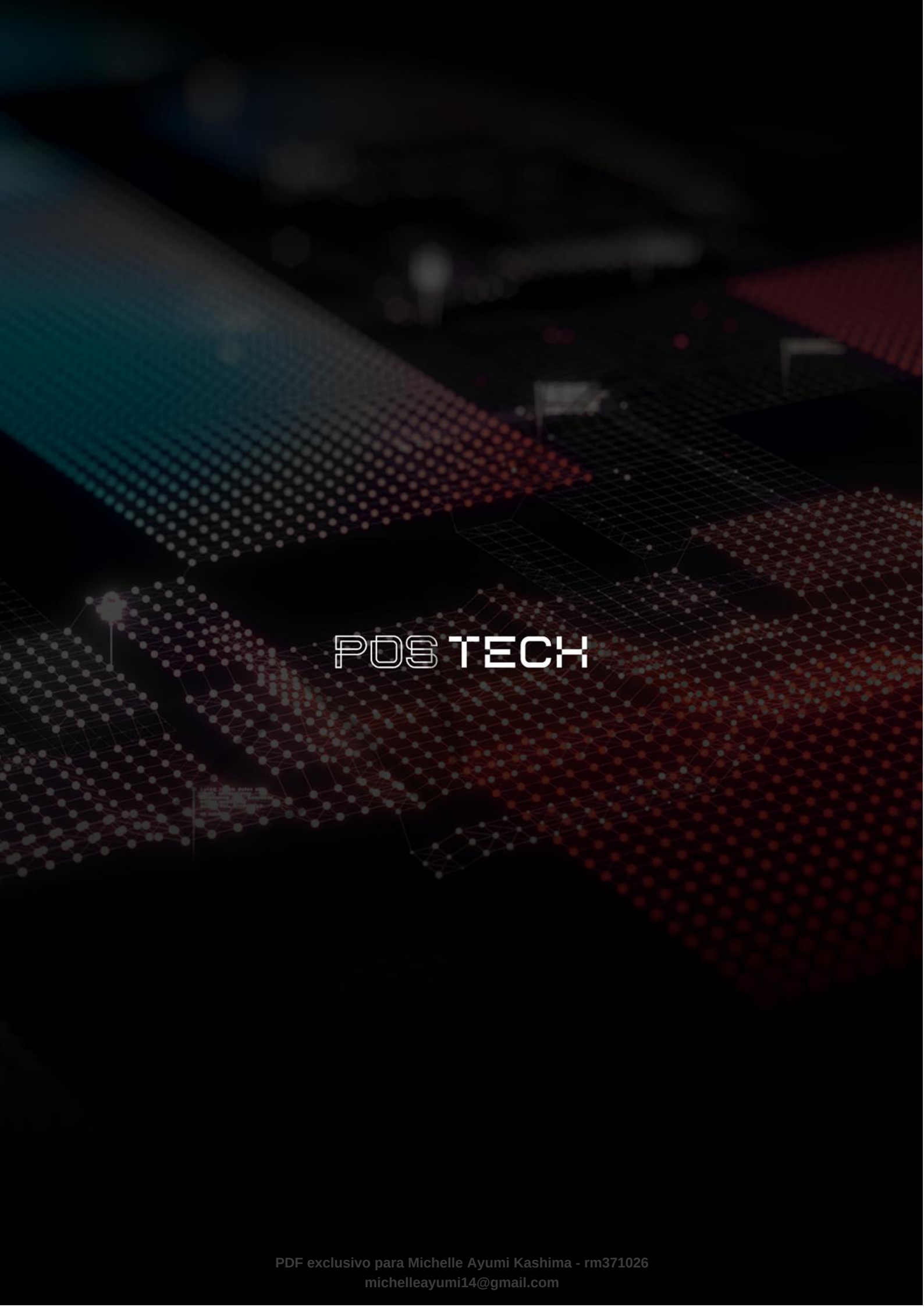
PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

Resiliência. Circuit Breaker. Resilience4j

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com